

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

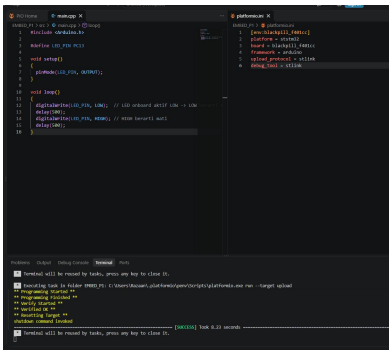
Alya Hasna Fadilah - 5024241044

2026

1 Langkah-Langkah Percobaan

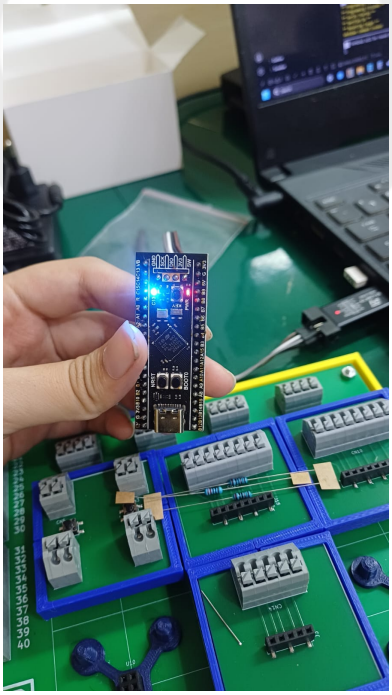
1.1 Percobaan Pertama: Pengenalan STM32 Blackpill

- Menyiapkan platformio.ini dan memasukkan kode sesuai dengan yang ada di modul
- menguhubgkan STM ke kabel lalu di cocokkan melalui USB sesuai dengan pin nya.
- Tulis kode blink di src/main.cpp lalu build dan upload



Gambar 1: Setup platformio.ini dan kode

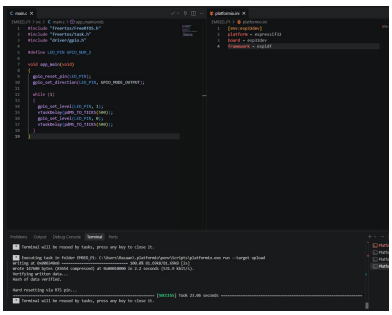
- LED akan nge-blink sesuai dengan program yang telah dijaklankan



Gambar 2: BLINK STM32

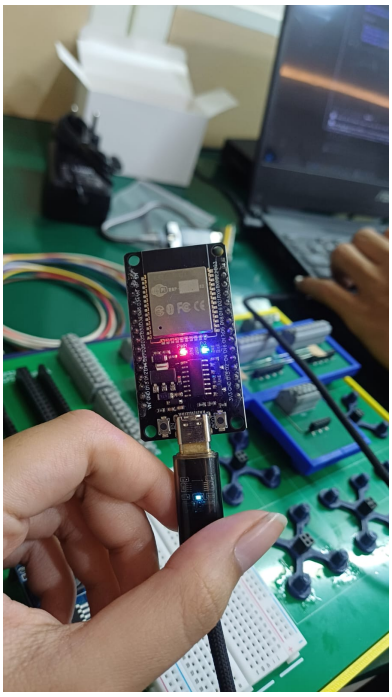
1.2 Percobaan Kedua: Pengenalan ESP32 dengan Frmework ESP-IDF

- Menyiapkan platformio.ini dan memasukkan kode sesuai dengan yang ada di modul
- menguhubgkan ESP32 ke kabel lalu di cocokkan melalui USB sesuai dengan pin nya
- Tulis kode blink di src/main.c lalu build dan upload



Gambar 3: Setup ESP-IDF dan kode

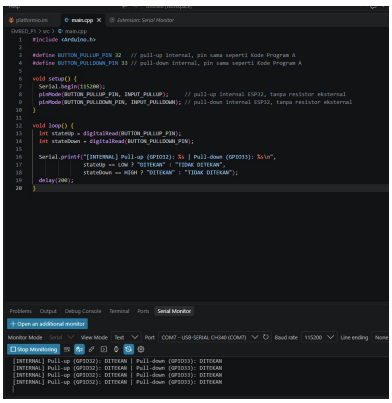
- LED akan nge-blink sesuai dengan program yang telah dijalankan



Gambar 4: BLINK ESP32

1.3 Percobaan Ketiga: Pull-up dan Pull-down: Eksternal vs internal

- Menyiapkan platformio.ini dan memasukkan kode sesuai dengan yang ada di modul
- Skema rangkaian bagian A, dan tulis kode program A pada src/main.cpp
- Tekan kedua tombol dan amati hasilnya

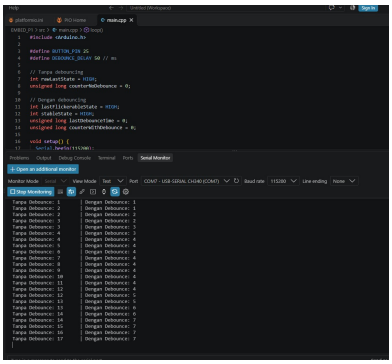


Gambar 5: Output

- Skema rangkaian bagian B
- ganti isi src/main.cpp dengan kode program B, amati hasilnya

1.4 Percobaan Keempat:Debouncing pada input tombol GPIO

- Menyiapkan platformio.ini dan memasukkan kode sesuai dengan yang ada di modul
- Membuat rangkaian sesuai dengan yang di modul
- Tekan tombol satu kali, lalu amati
- Ubah debounce delay menjadi 10 ms
- Ubah kembali menjadi 200 ms
- bandingkan ketiga nilai (10, 50, dan 200 ms)
-



Gambar 6: Output Debounce

1.5 Percobaan Kelima: Akses GPIO Analog: Pembacaan ADC dengan LDR

- Buat rangkaian sesuai dengan yang ada di modul
- tulis program yang membaca nilai ADC
- Build dan upload

- Tutup LDR degan tangan, allu sinari dengan lampu
- Analisis mengapa sinyal analog bias menunjukkan banyak kondisi.

2 Analisis Hasil Percobaan

2.1 Percobaan 1

Menguji mikrokontroller STM 32 Blckpill, berdasarkan percobaan hasil yang didapat adalah lampu nge-blink dimana menggunakan framework arduino dan ST-link untuk melakukan proses flashing. Sehingga lampu menyala dan mati secara bergantian. Dari percobaan tersebut menunjukkan bahwa arduino dapat mengakses GPIO STM32 dengan lebih sederhana.

2.2 Percobaan 2

Menguji mikrokontroller ESP32 menggunakan framework ESP-IDF sehingga menghasilkan output lampu nge-blink. Karena menggunakan ESP-IDF sehingga memiliki fungsi khusus untuk mengatur GPIO sehingga apabila LED berhasil berkedip secara bergantian, konfigurasi antara GPIO dan flashing telah berjalan dengan baik. Sehingga, ESP-IDF menunjukkan bahwa dapat memberikan kontrol yang lebih langusng terhadap perangkat keras dan memiliki program yang lebih kompleks daripada arduino.

2.3 Percobaan 3

Menguji pembacaan input digital menggunakan dua pushbutton. sehingga membandingkan hasil dengan menggunakan resistor pull-up dan pull-down, dengan yang ada di mikrokontroller. Dapat dilihat bahwa jika tidak menggunakan resistor eksternal tetap menghasilkan hasil yang sama yaitu jika tombol ditekan amak hasilnya LOW dan jika dilepas akan HIGh. Dua-duanya memiliki fitur resistor pull-up dan pull-down yang sama

2.4 Percobaan 4

Menguji bouncing pada pushbutton, ketika menekan tombol button maka akan menghasilkan sinyal HIGH dan LOW dalam waktu yang singkat. sehingga penghitung tanpa debouncing akan langsung bertambah jumlah hitungannya karena ada bouncing/ efek getaran. penghitung yang ada debouncing nya akan terhitung satu kali. Semakin kecil waktu debpunce nya maka semakin cepat tombol untuk merrespon sehingga gangguan bouncing bisa terjadi. Jika waktu debounce lebih besar dapat menyaring perubahan sinyal yang tidak diinginkan. Sehingga mendapatkan input digital yang lebih stabil. Tetapi pada praktikum ini terdapat kendala yaitu kurangnya waktu sehingga tidak dapat menyelesaikan sampai perbandingan antara 10 ms, 50 ms, 200 ms.

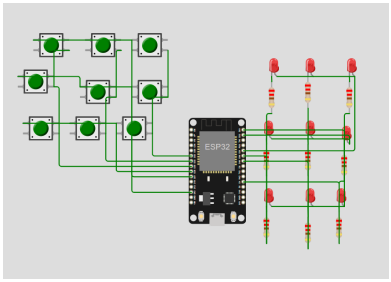
2.5 Percobaan 5

Menguji intensitas cahaya menggunakan sensor LDR dimana membaca tegangan untuk menggunakan fitu ADC menjadi digital. Ketika intensitas cahaya yang diterima LDRA berubah, maka nilai

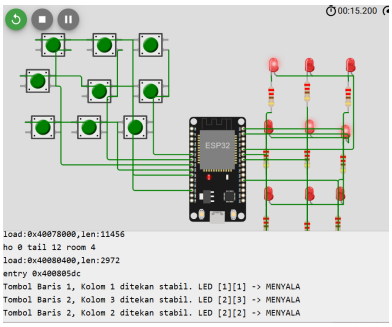
resistansi nya juga ikut berubah. sehingga dibaca oleh ADC dan dikonversi menjadi perkiraan tegangan sehingga semakin besar nilai ADC maka semakin besar tegangan yang terukur pada pin. Jika input digital hanya mengetahui dua kondisi yaitu HIGH dan LOW. sehingga pembacaan analog itu lebih memperoleh variasi nilai dan menunjukkan nilai perubahan intensitas cahaya yang bertahap. Ada beberapa kendala dikarenakan kurangnya waktu saat praktikum sehingga tidak dapat mencoba langsung percobaan kelima.

3 Hasil Tugas Modul

3.1 Jawaban Analisis



Gambar 7: Rangkaian



Gambar 8: Hasil Simulasi

```
#include <Arduino.h>

const int numRows = 3;
const int numCols = 3;
const int btnRowPins[numRows] = {13, 27, 26};
const int btnColPins[numCols] = {25, 33, 32};
const int ledRowPins[numRows] = {21, 22, 23};
const int ledColPins[numCols] = {19, 18, 4};

int buttonState[numRows][numCols];
int lastFlickerableState[numRows][numCols];
unsigned long lastDebounceTime[numRows][numCols];
const unsigned long debounceDelay = 50;

bool ledState[numRows][numCols];

void setup() {
  Serial.begin(115200);

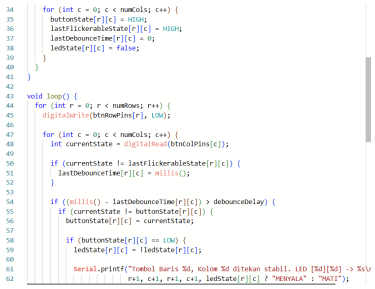
  for (int i = 0; i < numRows; i++) {
    pinMode(btnRowPins[i], OUTPUT);
    digitalWrite(btnRowPins[i], HIGH);

    pinMode(btnColPins[i], INPUT_PULLUP);
  }

  for (int r = 0; r < numRows; r++) {
    pinMode(ledRowPins[r], OUTPUT);
    digitalWrite(ledRowPins[r], LOW);

    pinMode(ledColPins[r], OUTPUT);
    digitalWrite(ledColPins[r], HIGH);
```

Gambar 9: Kode



```
  for (int c = 0; c < numCols; c++) {
    buttonState[r][c] = HIGH;
    lastFlickerableState[r][c] = HIGH;
    lastDebounceTime[r][c] = 0;
    ledState[r][c] = false;
  }
 }
}

void loop() {
 for (int r = 0; r < numRows; r++) {
  digitalWrite(btnRowPins[r], LOW);

  for (int c = 0; c < numCols; c++) {
   int currentState = digitalRead(btnColPins[c]);

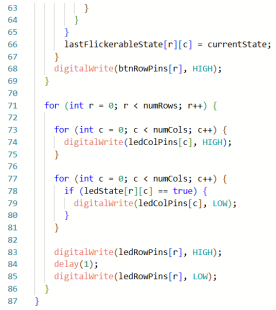
   if (currentState != lastFlickerableState[r][c]) {
    lastDebounceTime[r][c] = millis();
   }

   if ((millis() - lastDebounceTime[r][c]) > debounceDelay) {
    if (currentState != buttonState[r][c]) {
     buttonState[r][c] = currentState;

     if (buttonState[r][c] == LOW) {
      ledState[r][c] = !ledState[r][c];

      Serial.printf("Tombol Baris %d, Kolom %d ditekan stabil. LED [%d][%d] -> %s\r
                    r+1, c+1, r+1, c+1, ledState[r][c] ? "MENYALA" : "MATI");
```

Gambar 10: Kode



```
     }
    }
   }
   lastFlickerableState[r][c] = currentState;
  }
  digitalWrite(btnRowPins[r], HIGH);
 }

 for (int r = 0; r < numRows; r++) {

  for (int c = 0; c < numCols; c++) {
   digitalWrite(ledColPins[c], HIGH);
  }

  for (int c = 0; c < numCols; c++) {
   if (ledState[r][c] == true) {
    digitalWrite(ledColPins[c], LOW);
   }
  }

  digitalWrite(ledRowPins[r], HIGH);
  delay(1);
  digitalWrite(ledRowPins[r], LOW);
 }
}
```

Gambar 11: Kode

1. Bouncing berasal dari sifat mekanis tombol, Jadi jika kontak logam bertemu atau berpisah, pegasnya bergetar sehingga kontak terhubung dan terputus berulang kali selama beberapa milidetik,sehingga sinyal terbaca HIGH-LOW-HIGH-LOW. Debouncing memastikan satu tekanan fisik dihitung satu kali.

2. pull-up internal INPUT PULLUP. Saat tombol tidak ditekan, pin kolom berada pada HIGH lewat resistor pull-up internal. Saat baris aktif diberi LOW dan tombol ditekan, kolom tertarik ke GND sehingga terbaca LOW. Pin tidak boleh dibiarkan floating karena pin input berimpedansi sangat tinggi dan tidak punya level tegangan pasti.

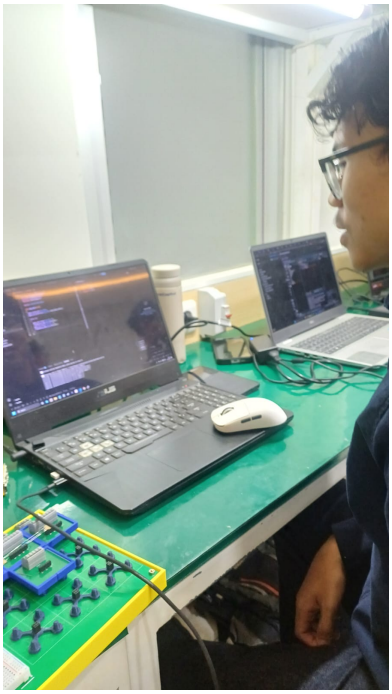
3. Jika satu tombol diganti LDR maka GPIO digital hanya membandingkan tegangan dengan ambang logika, ADC mengubah tegangan menjadi bilangan biner, LDR merubah hambatannya menyesuaikan dengan intensitas cahaya. Tegangan yang dihasilkan akan bervariasi kontinu

4 Kesimpulan

Kesimpulannya adalah berdasarkan lima percobaan, dapat diketahui bahwa mikrokontroller STM32 Blackpill dan ESP 32 memiliki kemampuan mengendalikan serta membaca sinyal melalui GPIO. Percobaan pertama dan kedua memperkenalkan penggunaan GPIO sebagai output digital melalui program Blink, sekaligus menunjukkan perbedaan struktur pemrograman antara framework Arduino dan ESP-IDF. Percobaan ketiga menunjukkan penggunaan GPIO sebagai input digital dengan konfigurasi pull-up dan pull-down, baik secara eksternal maupun internal. Percobaan keempat membuktikan pentingnya penerapan debouncing untuk mengatasi ketidakstabilan sinyal akibat karakteristik mekanis tombol. Sementara itu, percobaan kelima memperkenalkan penggunaan GPIO sebagai input analog melalui ADC untuk membaca perubahan tegangan dari sensor LDR. Terdapat beberapa kendala selama praktikum yaitu dikarenakan waktu yang terbatas sehingga tidak dapat menyelesaikan praktikum 4 dan 5.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 12: Dokumentasi Praktikum